# Lab Session – 01
# GIT Version Control

# Core Architecture: Git's Three Areas

## 1. Working Directory

The local filesystem where you actively write, edit, and experiment with code.

Changes here remain **untracked or modified** until specifically added to the staging index.

## 2. Staging Area (Index)

A preparation draft zone where modifications are organized via git add.

Allows developers to craft precise, **atomic commits** before taking a snapshot.

## 3. Repository (.git)

The permanent object database containing immutable commit snapshots created via git commit.

Pointed to by HEAD, capturing the full project evolution across time.

# Essential Git Workflow & Lifecycle

- **Initializing & Remote Sync:** Start new tracking with git init. Connect to team servers using git remote add origin to enable collaborative pushes and pulls.
- **Ignoring Unwanted Files:** Configure .gitignore early to exclude build artifacts, dependencies (e.g., node_modules/), binaries, and secret credentials.
- **Atomic Committing:** Treat each logical feature or fix as a separate commit. Stage files selectively and write clear commit messages explaining *why* changes were made.
- **importance of Version Control:** Enables seamless multi-developer teamwork, provides effortless historical reverts, and preserves project integrity over time.

# Branching & Merging

## Feature Branching Protocol

Isolate feature work from stable production code using dedicated branches:

`git checkout -b feature`

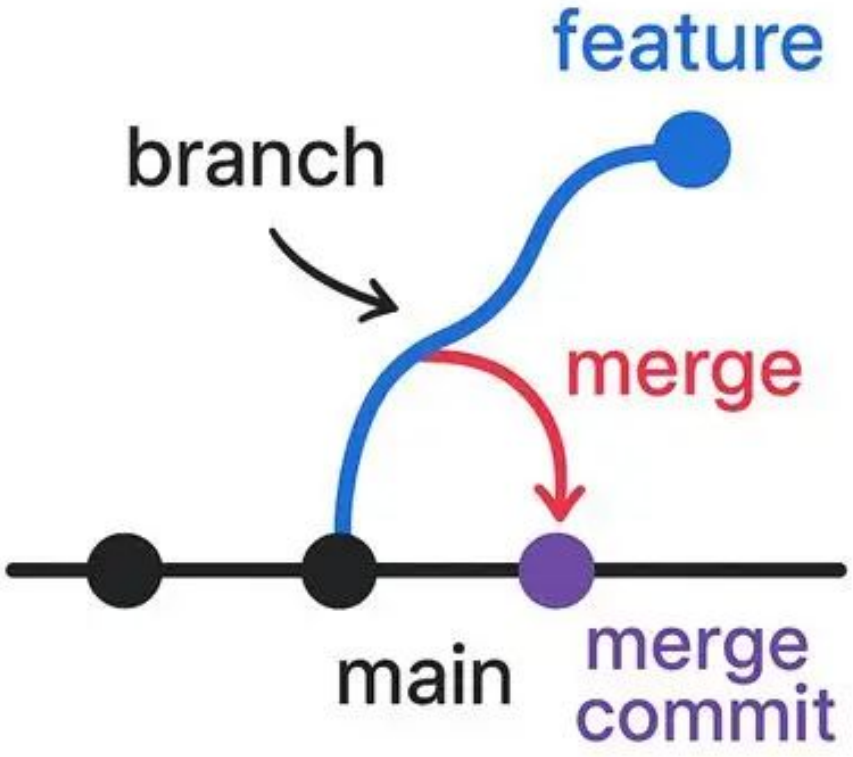
Prevents incomplete experiments from breaking the primary build on main.

## Integrating Code

Merge completed features back cleanly:

`git merge feature`

Git executes either a **Fast-Forward** update or a **3-way Merge Commit** based on history divergence.



# File Operations & History Lineage

## Renaming & Deleting Files

**Renaming & Moving:** Avoid manual deletion. Use git mv old_path new_path so Git explicitly tracks content lineage without breaking commit history.

**Deleting Tracked Files:** Remove files cleanly via git rm filename to automatically stage deletion for the upcoming commit snapshot.

## Restoring & Reverting State

**Restoring Modified Content:** Discard unstaged working tree edits cleanly via git restore without impacting tracked repository history.

**Selective File Reversion:** Checkout an exact historical state of a single file using git checkout -- .

# History Investigation & Inspection

## git log

Visualize commit lineages and tags across all branches in a compact graphical view:

git log --graph --oneline --all

## git blame

Examine file lines individually to identify author, timestamp, and specific commit SHA:

git blame filename.js

## git diff

Inspect line additions and deletions between working directory, staging, or branches:

git diff main..feature

# Recovery Techniques & Safety Nets

## Amending & Resetting HEAD

**Amending Commits:** Fix immediate typos or add forgotten staged files to your last commit via git commit --amend.

**Resetting HEAD:** Use git reset --soft to undo commit snapshots while keeping working edits, or --hard to discard local changes.

## Git Reflog Safety Net

**The Reflog Mechanism:** git reflog logs every local HEAD pointer move across time.

**Accident Recovery:** Provides a safety mechanism to recover detached HEADs or accidentally deleted local commits and branches.

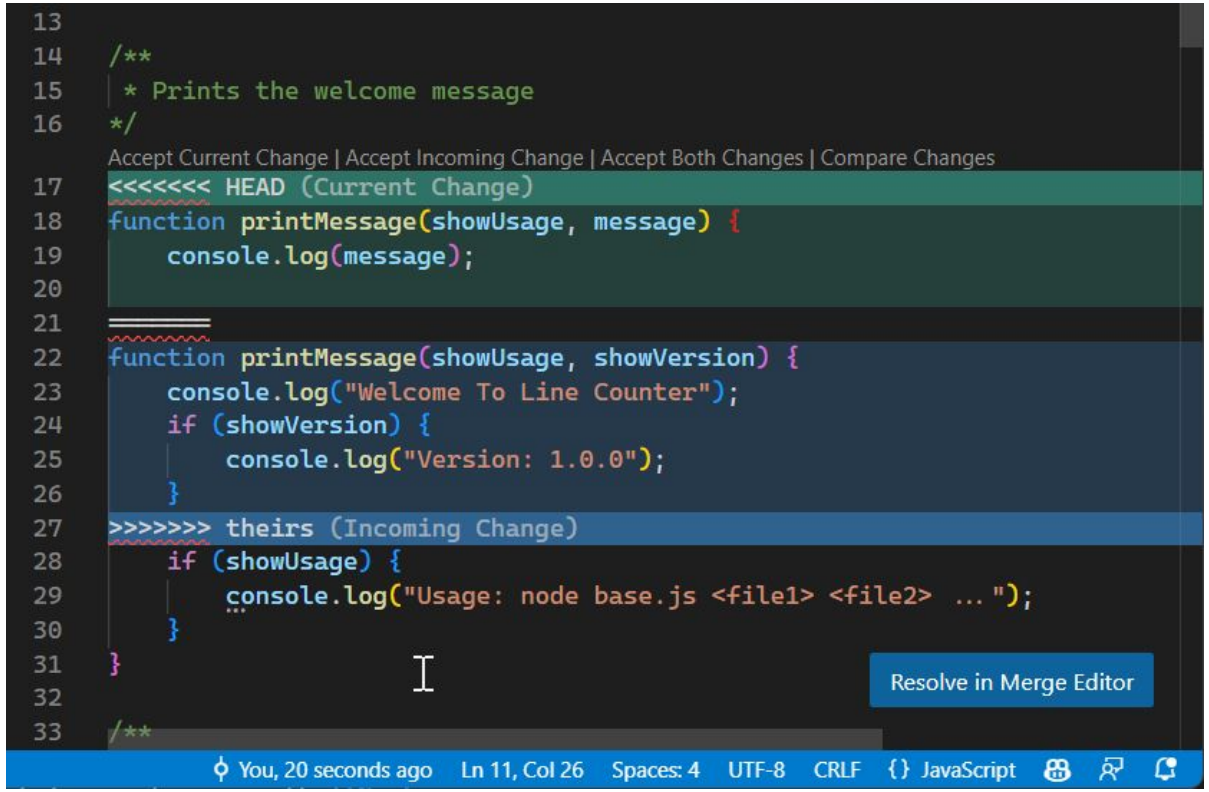
# Resolving Merge Conflicts

## Understanding Divergence

Conflicts occur when Git attempts to merge overlapping modifications on identical lines across concurrent branches.

## Manual Resolution Steps

- Locate markers: <<<<<<<, =======, >>>>>>>.
- Edit file content manually to retain the intended final code state.
- Stage resolved files via git add and finalize with git commit.



# Git Tool Cheat Sheet

| Command | Primary Purpose | Safety & Best Practice |
| --- | --- | --- |
| git stash | Temporarily shelves uncommitted modifications to switch tasks cleanly. | Apply changes back via git stash pop when ready. |
| git cherry-pick | Applies a specific commit from another branch onto current branch. | Use sparingly to avoid creating duplicate commit histories. |
| git rebase | Linearizes history by reapplying commits on top of another base. | **Never** rebase shared public branches! |
| git tag -a v1.0 | Marks specific points in repository history as official release milestones. | Push tags explicitly using git push origin --tags. |